

QR Code Registration: dao_client + GAS Implementation Plan

Date: 2026-06-12

Author: Sophia (TrueSight Autopilot)

Status: Draft — awaiting governor approval

1. Problem Statement

There is no way to register a single Agroverse QR code from the autopilot or dao_client. The existing flow requires:

- Adding a row to the "Agroverse QR codes" Google Sheet manually
- Running the batch compiler from a macOS machine with the right fonts and templates

This means the autopilot cannot mint a QR code for an event (like SF Tech Fest) without manual sheet editing. The gap is: **no GAS endpoint for single QR registration** and **no dao_client command to call it**.

2. Solution Architecture

...

dao_client register_qr_code



Edgar (sentiment_importer)

■ POST /dao/qr_code_register

■ (signed contribution → Telegram Chat Logs)



GAS Web App (process_qr_code_generation_telegram_logs.gs)

■ Reads Telegram Chat Logs

■ Creates row in "Agroverse QR codes" sheet

■ Triggers GitHub Actions webhook



GitHub Actions (lineage-assets)

■ Runs batch_compiler.py

■ Generates branded QR PNG

■ Commits PNG + JSON manifest



QR code live at truesight.me/qr/?id=<code>

...

Component Breakdown

Component	Repo	What it does
dao_client command	`dao_client`	New `register_qr_code` CLI command. Signs a `[QR CODE REGISTRATION]` event, submits to Edgar.
Edgar endpoint	`sentiment_importer`	New `DaoController#register_qr_code` action. Receives signed event, appends to Telegram Chat Logs.
GAS processor	`tokenomics` (GAS script `1N6o00N9VtRK...`)	New `processQrCodeRegistrationTelegramLogs()` function. Reads `[QR CODE REGISTRATION]` messages, creates single row in Agroverse QR codes sheet.
GitHub Actions	`lineage-assets`	Existing `generate_qr_batch.sh` workflow. Triggered by GAS webhook. Generates branded QR PNG + manifest.

3. Implementation Phases

Phase 1: GAS Endpoint (Single QR Registration)

PR1 — `tokenomics` — Add `register\_single\_qr\_code.gs`

New GAS file that adds a `doGet(e)` handler for `?action=registerSingleQRCode` with parameters:

- `qr\_code` (string) — the QR code ID (e.g. `SFTF\_FR\_20260612\_1`)
- `landing\_page` (string) — the URL the QR points to
- `farm\_name` (string) — display name
- `state` (string)
- `country` (string)
- `year` (string) — harvest year
- `currency` (string) — product name
- `status` (string) — defaults to `SAMPLE`
- `manager` (string) — who minted it
- `creation\_date` (string) — YYYYMMDD

Logic:

1. Validate required fields
2. Check for duplicate `qr\_code` in sheet
3. Append row to "Agroverse QR codes" tab
4. Trigger GitHub Actions webhook for single QR generation
5. Return success/error JSON

Deployment: clasp mirror `1N6o00N9VtRK\_L3e0NQXEsmC6QME1KObZdmdbJgo0Tbgj\_7P-EINL5THn`

Phase 2: Edgar Endpoint

PR2 — `sentiment\_importer` — Add `DaoController#register\_qr\_code`

New action that:

1. Accepts POST with signed `[QR CODE REGISTRATION]` event
2. Verifies digital signature
3. Appends to "Telegram Chat Logs" sheet
4. Triggers `WebhookTriggerWorker` → GAS `processQrCodeRegistrationTelegramLogs`

Phase 3: dao_client Command

PR3 — `dao\_client` — Add `register\_qr\_code` CLI command

```
```bash
truesight-dao-register-qr-code \
--qr-code "SFTF_FR_20260612_1" \
--landing-page "https://agroverse.shop/friends-of-the-rainforest" \
--farm-name "SF Tech Fest" \
--state "CA" \
--country "USA" \
--year "2026" \
--currency "Friends of the Rainforest" \
--status "SAMPLE" \
--manager "Gary Teh"
```
```

Logic:

1. Build `[QR CODE REGISTRATION]` event payload
2. Sign with contributor key
3. POST to Edgar `/dao/qr\_code\_register`

4. Print result + QR code URL

Phase 4: Autopilot Integration

PR4 — `truesight\_autopilot` — Wire autopilot to use new command

Update the autopilot's QR code workflow so that when a governor asks to mint a QR code:

1. Call `dao\_client register\_qr\_code` with the parameters
2. Wait for the GAS + GitHub Actions pipeline
3. Report the QR code URL back

4. Gates

| Gate | Phase | Description | Who |
|------|---------|--|------|
| G1 | Phase 1 | GAS deploys, manual test with curl | Gary |
| G2 | Phase 2 | Edgar endpoint deployed, test with signed request | Gary |
| G3 | Phase 3 | dao_client command works end-to-end | Gary |
| G4 | Phase 4 | Autopilot successfully mints a QR for SF Tech Fest | Gary |

5. UAT (User Acceptance Tests)

U1: GAS Single QR Registration

1. `curl` the GAS web app with `?action=registerSingleQRCode&qr\_code=UAT\_TEST\_001&...`
2. Verify row appears in "Agroverse QR codes" sheet
3. Verify status is `MINTED`

U2: Duplicate Detection

1. Register the same QR code twice
2. Verify second call returns error "QR code already exists"

U3: Edgar Endpoint

1. POST signed `[QR CODE REGISTRATION]` to Edgar
2. Verify row appears in "Telegram Chat Logs"

3. Verify GAS processes it and creates sheet row

U4: dao_client Command

1. Run ``truesight-dao-register-qr-code --dry-run``
2. Verify output shows correct payload
3. Run without ``--dry-run``
4. Verify QR code appears in sheet

U5: End-to-End: SF Tech Fest QR

1. Run `dao_client` to register ``SFTF_FR_20260612_1``
2. Wait for GitHub Actions to generate branded PNG
3. Verify PNG at ``lineage-assets/pngs/SFTF_FR_20260612_1.png``
4. Verify JSON manifest at ``lineage-assets/qrs/SFTF_FR_20260612_1.json``
5. Verify ``https://truesight.me/qr/?id=SFTF_FR_20260612_1`` resolves

U6: Error Handling — Missing Fields

1. Call endpoint without required fields
2. Verify descriptive error message

U7: Error Handling — Invalid Status

1. Call with invalid status value
2. Verify validation error

U8: Batch Compatibility

1. Verify existing batch QR generation still works
2. Verify no regression in ``processQRCodeGenerationTelegramLogs``

U9: Autopilot Integration

1. Ask autopilot to "mint a QR code for SF Tech Fest"
2. Verify autopilot calls `dao_client`
3. Verify QR code is generated

6. Files to Create/Modify

New Files

| File | Repo | Purpose |
|---|----------------------|---|
| `google_app_scripts/agroverse_qr_codes/register_single_qr_code.gs` | `tokenomics` | GAS endpoint for single QR registration |
| `dao_client/commands/register_qr_code.py` | `dao_client` | CLI command |
| `sentiment_importer/app/controllers/qr_code_registration_controller.rb` | `sentiment_importer` | Edgar endpoint |

Modified Files

| File | Repo | Change |
|--|-----------------------|-----------------------------------|
| `dao_client/setup.py` | `dao_client` | Register new command entry point |
| `sentiment_importer/config/routes.rb` | `sentiment_importer` | Add route for QR registration |
| `truesight_autopilot/app/qr_workflow.py` | `truesight_autopilot` | Wire autopilot to use new command |

7. Risks & Mitigations

| Risk | Likelihood | Mitigation |
|-------------------------------------|------------|---|
| GAS web app quota exceeded | Low | Single QR registration is lightweight |
| GitHub Actions webhook fails | Medium | GAS retries with exponential backoff |
| Duplicate QR code IDs | Low | Sheet-level unique check in GAS |
| Font/Helvetica missing on autopilot | Medium | Install fonts or use fallback in batch_compiler |

8. Timeline (Estimated)

| Phase | Effort | Dependencies |
|---------------------|------------------|-----------------------------------|
| Phase 1: GAS | 2-3 hours | clasp access to `1N6o00N9VtRK...` |
| Phase 2: Edgar | 1-2 hours | sentiment_importer deploy |
| Phase 3: dao_client | 1-2 hours | dao_client release |
| Phase 4: Autopilot | 1 hour | truesight_autopilot deploy |
| Total | 5-8 hours | |

9. RESUME HERE

When governor says "go for it":

1. Open PR1: Create ``register_single_qr_code.gs`` in ``tokenomics/google_app_scripts/agroverse_qr_codes/``
2. Deploy GAS via clasp mirror
3. Open PR2: Add Edgar endpoint in ``sentiment_importer``
4. Open PR3: Add `dao_client` command
5. Open PR4: Wire autopilot
6. Run UAT U1–U9
7. Report results in this thread

Gate: PRs are opened only — NEVER self-merge. Governor reviews and merges.